



NOWITNESS

Security Audit Report

Upgradability Assessment

Date	September 24, 2026
Client	Cardano Foundation
Project	CIP-113 Programmable Tokens
Version	0.0.1

Contents

Disclosure	1
Disclaimer	2
Scope	4
Assessment Overview	5
Assessment Components	6
Executive Summary	7
Code Base	8
Severity Classification	10
Finding Severity Ratings	11
Findings	12
[L 201] Missing Datum-Size Check During Issuance	13
[I 101] Coordination ADA Ratchet Locks Excess Lovelace	15
[I 102] Fixed Protocol Action Set Limits Upgradability	17
[I 103] Redundant Fields in Protocol Params	19
[I 104] Redundant nonce parameter in coordination_spend validator	20
[I 105] Static Upgrade-Multisig Configuration	21
[I 106] Fixed Registry Lifecycle Limits Node-Schema Upgrades	23
[I 107] Fixed Issuance Policies Limit Upgrades for Existing Tokens	25

Disclosure

This document contains proprietary information belonging to No Witness Labs. Duplication, redistribution, or use, in whole or in part, in any form, requires prior written consent from No Witness Labs.

Nonetheless, both the customer **Cardano Foundation** and No Witness Labs are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the Protocol.

Disclaimer

This review reflects the state of the codebase at the time of assessment and should be regarded as a point-in-time evaluation. Any changes made after the assessment period fall outside the scope of this report.

Although every effort was made to identify potential risks and vulnerabilities, no security review can guarantee the discovery of all issues. The information and recommendations in this report are provided solely for technical purposes and must not be considered investment advice.

To strengthen security assurance, projects are encouraged to seek multiple independent reviews and to supplement audits with practices such as bug bounty programs.

The scope of this audit was limited to the code provided. It did not extend to compiler-level artifacts (e.g., generated UPLC code) or to areas beyond the explicitly reviewed codebase. Additional testing methodologies, including unit testing and property-based testing, were also outside the scope of this engagement.

Trusted Initialization and Deployment

The security properties described in this report assume that the canonical deployment is correctly initialized and verified before use. This includes the intended validator instances and parameters, script hashes, minting-policy identifiers, protocol reference UTxOs and their identifying assets, issuance script-template data, initial protocol parameters, and initial upgrade-authority configuration, including the applicable authorization rules.

Deployment operators and parties accepting the canonical deployment are responsible for verifying this configuration. Successful on-chain initialization does not, by itself, establish that all configured values identify the intended implementations or constitute a safe operational configuration. Incorrect initialization or reliance on a non-canonical deployment may invalidate the applicable conclusions of this assessment.

Trusted Protocol Upgrade Authority

The authority identified by the current protocol-parameters datum's `upgrade_cred` is assumed to be securely controlled and trusted to approve only intended, correctly configured, and compatible upgrades. This responsibility includes selecting replacement validators, setting configurable limits, and maintaining a usable authorization mechanism when updating or replacing the upgrade authority.

This authority represents an ongoing, deployment-wide trust dependency. It can replace the delegated validation logic governing existing programmable-token UTxOs without changing their `programmable_logic_base` addresses. A malicious or erroneous authorized upgrade may therefore remove ownership, per-policy authorization, or custody protections supplied by the replaceable validators, potentially permitting asset movement without holder consent or causing loss of funds, availability, or future upgradability.

The conclusions of this report apply to the implementations and configuration assumptions reviewed. They do not automatically extend to subsequent validator replacements or configuration changes. An unchanged protocol-parameters NFT or PLB address does not establish that the active validation logic remains unchanged or covered by this assessment.

These assumptions concern the correctness of deployment configuration and the authority's upgrade choices. They do not exclude from review the on-chain mechanisms that authenticate configuration, enforce upgrade authorization, or preserve mandatory invariants. Bypassing those mechanisms remains a security issue even when initialization and upgrade decisions are trusted.

Scope

In scope

- On-chain validators and supporting logic at the time of review.
- Datum/redeemer structures and on-chain invariants (value, units/rates, cross-phase checks).
- Use of reference UTxOs for configuration and script parameters.

Out of scope

- Compiler layer / UPLC output and toolchain internals.
- Creation of new unit tests or property-based tests.
- Off-chain libraries, front-end apps, wallet UX (incl. blind-signing), unless explicitly constrained on-chain.
- Third-party protocol internals (bridges/liquidity venues), centralized services, exchanges, oracles, external price feeds.
- Node operations, networking, mempool policy variance, ordering/MEV-like effects.

Assessment Overview

From **August 25, 2026** to **September 08, 2026**, **Cardano Foundation** engaged No Witness Labs to conduct a comprehensive security assessment of the **CIP-113 Programmable Tokens** protocol. The evaluation was performed in accordance with established industry best practices for code auditing and protocol analysis.

The audit followed a structured five-phase process:

- **Planning** – Defined the assessment scope and objectives, and aligned expectations with the client.
- **Discovery** – Acquired an understanding of the protocol through client discussions, documentation review, and preliminary analysis of architecture and design.
- **Manual Revision** – Conducted an in-depth review of the codebase to identify potential vulnerabilities, design weaknesses, and security risks.
- **Validation** – Verified client-provided fixes and obtained acknowledgments for resolved issues.
- **Reporting** – Documented all confirmed findings, assigned severity levels, and provided actionable recommendations for remediation.

The team also evaluated protocol efficiency and potential optimizations. Findings were prioritized by severity level.

Assessment Components

Manual revision

Our manual code auditing is focused on a wide range of attack vectors, including but not limited to:

- UTXO Value Size Spam (Token Dust Attack)
- Large Datum or Unbounded Protocol Datum
- EUTXO Concurrency DoS
- Unauthorized Data modification
- Multisig PK Attack
- Infinite Mint
- Incorrect Parameterized Scripts
- Other Redeemer
- Other Token Name
- Arbitrary UTXO Datum
- Unbounded protocol value
- Foreign UTXO tokens
- Double or Multiple satisfaction
- Locked Ada
- Locked non Ada values
- Missing UTXO authentication
- UTXO contention

Executive Summary

Project Overview

CIP-113 (Programmable Tokens) defines a standard for Cardano native tokens whose ownership changes require successful script validation. Programmable tokens are held at addresses that use `programmable_logic_base` (PLB) as the payment credential, while the stake credential identifies the owner.

This assessment focuses on the upgradability feature of the CIP-113 implementation. The feature allows the protocol's upgrade authority to replace the standard `transfer`, `third_party`, and `unfracking` validators through protocol-parameter updates. Existing programmable-token UTxOs can then use compatible replacement implementations without changes to their PLB addresses or token migration.

Code Base

Repository

<https://github.com/cardano-foundation/cip113-programmable-tokens>

Branch

main

Audited Commit

9db7e0629a1509cc9d41d069f0ef0ed251601173

Fixes Verified Commit

6b75ba3286b4692ca23059ff51285db357fb09c6

Files Audited

Files	SHA256 Checksum
env/default.ak	1501134ad4d911618e4fadc350c5e2de49e4b71672c1af5810a9ca67045277fc
lib/assets.ak	c038cf403cd8244ee45bc9f5eae98c1ae6583c7c71d43747eadead028c0f7a34
lib/linked_list.ak	769c218a232f22614a64a77c1913c52459dcc2fc0f7ed8c4272cde334ee99a2c
lib/list.ak	01c1bd920f0f18769a09e8dddb2cd493bd43e2dddec3df8c00749d3e777435d7
lib/pairs.ak	f61ffb48c246c24c3fa15e130677eee205dfecaba93644e8b017369b01b52175
lib/registry_node.ak	39f1e555cd8fdd117793908f4d569991a8ba723cd17f558ce7dbaf58d6fe696e
lib/tokens.ak	185f68523f40fa917177fe89444cbfe7d963b94b69b385da855c6b4aa4939ace
lib/types.ak	0fbfe9ba710d8662050afbb3a90ee9189cc52aafa4966593273aa9727a0d1f5a
lib/utls.ak	d79180c027d6c1112cf499b26e51eec68c5da4f3c1034729fca2708dc083b3d4
validators/always_fail.ak	3388ade147c1349a7dcf1ca2b045d7ff159a8e8fa01550c52fc8874b8631106a
validators/coordination_spend.ak	ddc62750a81f460098ad086b7602c3ecc07d3a6d0880c2b48b67c6aab4e7b7bd
validators/issuance_cbor_hex_mint.ak	4027ad2fa2b6d51d18496917f0897868fffc73a7c051f791d581989ff310186e

Files	SHA256 Checksum
validators/issuance_mint.ak	eb9a07bafcdcd004fae48f8146981b799c95af083b5f4002606f37c62b0e982b
validators/programmable_logic/owner.ak	0bbb0667fd851bee61bd82665da8cfbcf3787a426f2379f208acde8eac7c7faf
validators/programmable_logic/params.ak	5e8efae670af4d20f956d5ed11a034167f46408ee3344f3f8e0e90e5ff12c9a4
validators/programmable_logic/third_party.ak	a42476298e0d6f55d388fa3cc63d49af5be878639e0829fe851b5051541d3f27
validators/programmable_logic/transfer.ak	7ab87eec2ec1e1176692add8678ccda6c5b8f6e1467d79df549a81ca9c9dc9fa
validators/programmable_logic/unfracking.ak	19b26b7fbc8ec383d9547a89573c826192b98636aad631d7713c5cb06e3b8df9
validators/programmable_logic_base.ak	3cbbea25cde26cc70d71de0889d1550a5c0cec5306b926c1594e598f11bc6ab3
validators/protocol_params_mint.ak	ad9736c4c2e1b9792a07a98c71abdd3063b9f1d68a70ab230f246a789c87d13f
validators/registry_mint.ak	cfed92d537fe72c66f7547ff04e487249f7894adbd1a126f14e151890f325681
validators/registry_spend.ak	cd1dfe11a0738f4d448fdd070d85318cd14e1c3a1f9272ae19c57b698dd3257c
validators/third_party.ak	51978e208cd264c5669d142c2382c64af9f7dd3a65046dcd9b3d9f07b425e62e
validators/transfer.ak	f953ac6dfc07a99f34e922bc1700d2299373571cb887368aaffff1e9f2f9b5441
validators/unfracking.ak	e2950843dee1ecd4d7445b4a696fd68c679c7108de5b7dc93681eb4391d1602d
validators/upgrade_multisig.ak	507007130b59b07dd3f3c4f440cd7fb4d36b8bb3039874c67910e0416413f28e

Severity Classification

- **Critical:** Vulnerabilities that enable direct theft of assets, permanent loss of funds, or catastrophic protocol failure. These represent immediate and severe financial risk with straightforward exploitation paths.
- **High:** Vulnerabilities that can result in financial loss or significant harm to users or the protocol, though impact may be limited to specific functions or require certain conditions. Exploitation paths exist but may be more complex than Critical Vulnerabilities.
- **Medium:** Vulnerabilities that do not directly result in financial loss but may temporarily impair functionality or create exploitable conditions. These are typically limited to state manipulations and include issues such as front-running susceptibility or logic flaws that compromise transaction integrity.
- **Low:** Low-impact Vulnerabilities with limited attack vectors, minimal attacker incentive, or requiring highly specific scenarios. These typically do not result in financial loss or significant harm to users or the protocol.
- **Informational:** Findings without immediate security or financial implications. These include optimization opportunities, code quality improvements, inconsistencies in naming conventions, design suggestions, and gaps in documentation.

Impact–Likelihood Severity Matrix

	High likelihood	Medium likelihood	Low likelihood
High impact	C	H / M	M
Medium impact	H	M	M / L
Low impact	M	M / L	L

Finding Severity Ratings

The following table defines levels of severity and score range that are used throughout the document to assess vulnerability and risk impact

	Level	Severity	Count
	5	Critical	0
	4	High	0
	3	Medium	0
	2	Low	1
	1	Informational	7
Total Findings			8

Findings

ID	Title	Severity	Status
201	Missing Datum-Size Check During Issuance	Low	Resolved
101	Coordination ADA Ratchet Locks Excess Lovelace	Informational	Resolved
102	Fixed Protocol Action Set Limits Upgradability	Informational	Resolved
103	Redundant Fields in Protocol Params	Informational	Resolved
104	Redundant nonce parameter in coordination_spend validator	Informational	Resolved
105	Static Upgrade-Multisig Configuration	Informational	Resolved
106	Fixed Registry Lifecycle Limits Node-Schema Upgrades	Informational	Acknowledged
107	Fixed Issuance Policies Limit Upgrades for Existing Tokens	Informational	Resolved

[L 201] Missing Datum-Size Check During Issuance

	Level	Severity	Status
	2	Low	Resolved

Category: Liveness

Affected File: `validators/issuance_mint.ak`

Description

When `issuance_mint` validates PLB output placement itself, its `no_escape` function uses `is_seizable_output_shape`, which rejects datum hashes and reference scripts but does not apply `max_inline_datum_bytes`.

The later PLB actions treat these outputs differently:

- **Transfer:** The core transfer path does not apply the size bound to input datums, but applies it to every PLB output.
- **Third-party:** Paired outputs must preserve their datums, while fresh destination outputs are bounded.
- **Unfracking:** Paired outputs preserve their datums and remove the acted policy; destinations receiving that policy are bounded.

Transfer also applies the bound to PLB outputs containing only newly minted policies, even though purely minted policies are excluded from its proof walk. A standalone mint accepted by `issuance_mint` can therefore fail when composed with an unrelated transfer. This rejection is the transfer validator enforcing its intended output bound; the inconsistency is that `issuance_mint` allows the same over-bound state to be created in a standalone transaction.

Impact

- **If the substandard scripts allow the datum to be changed or detached:** Ordinary transfer can continue with a bounded output datum. Authorized third-party and unfracking actions can also move the token to a bounded fresh output. These paths can remain available, but the oversized metadata or state cannot remain attached to the token.
- **If the substandard scripts require the datum to remain attached:** Ordinary transfer is blocked because every PLB output is bounded. A third-party action may keep the token and oversized datum together in its paired output, but cannot move them together to a fresh destination. Unfracking removes the token from its paired output, and its destination is bounded, so it cannot preserve the oversized datum with the token. These restrictions remain until the substandard requirement changes or `max_inline_datum_bytes` is raised.

Severity Rationale

The finding is rated Low despite its High impact because `issuance_mint` requires the token-specific minting-logic script to be invoked for every mint, allowing the issuing side to enforce the datum-size limit.

Recommendation

Apply the live `max_inline_datum_bytes` limit from authenticated protocol parameters when validating PLB output placement during issuance. Enforce the check in the component that owns issuance-level PLB output validation:

- **Current design:** Add the check to `issuance_mint`.
- **If Finding F107 is implemented:** Add the check to the upgradeable issuance-logic script that performs the delegated custody and PLB output checks, rather than to the permanent issuance policy.

Resolution

`issuance_logic` now enforces the serialized inline-datum size bound on outputs carrying the issued policy when it validates PLB output placement itself. The limit is supplied as a validator parameter and must be configured consistently with the transfer, third-party, and unfracking validators. Deployed parameter values were not verified.

[I 101] Coordination ADA Ratchet Locks Excess Lovelace

	Level	Severity	Status
	1	Informational	Resolved

Category: Design Limitation / Value Handling

Affected File: `validators/coordination_spend.ak`

Description

The `coordination_spend` validator requires the continuing coordination output to contain at least as much lovelace as the spent input. The coordination UTxO holds the protocol-parameters NFT and the live `ProgrammableLogicGlobalParams` datum, while the current datum's `upgrade_cred` authorizes updates.

Relevant code snippet from `validators/coordination_spend.ak`:

```
1 // ADA may only grow (top-up allowed, drain forbidden), so a min-UTxO
2 // parameter rise can never strand the coordination UTxO.
3 expect assets.expect_match(cont.value, own_input.output.value, >=)?
```

Aiken

The stdlib `assets.expect_match` helper passes the first value's lovelace as the comparator's first argument and the second value's lovelace as its second argument, then requires exact equality for all remaining assets. Because `cont.value` is first and the spent input value is second, `>=` enforces:

```
1 continuing-output lovelace >= input lovelace
```

text

This permits equal value and top-ups while rejecting every reduction, even when the protocol-parameters NFT and all other assets remain unchanged. The check is a structural constraint independent of the separate `upgrade_cred` authorization, so the current upgrade authority cannot authorize a lower-lovelace output. The source describes this one-way ratchet and the resulting unrecoverable overfunding as an explicit v1 policy choice. `ProgrammableLogicGlobalParams` contains no configurable minimum-lovelace target.

Scenario

1. The coordination UTxO receives a lovelace top-up.
2. A lower lovelace amount later becomes otherwise valid, creating a surplus relative to that amount.
3. The current upgrade authority submits an update that returns the protocol-parameters NFT and all other assets unchanged but reduces the lovelace amount.
4. `assets.expect_match` rejects the transaction because the continuing output contains less lovelace than the input.

This scenario requires both an actual surplus and an otherwise-valid lower output amount. No deployed surplus amount or economic value has been established.

Impact

Any such surplus cannot be recovered through the current coordination spend path, even by the current upgrade authority. The affected amount is limited to the difference between the coordination input's lovelace and the lower amount that would otherwise be valid.

Recommendation

Remove the relative lovelace constraint while retaining exact equality for non-ADA assets:

Minimal implementation:

```
1 expect assets.expect_match(  
2   cont.value,  
3   own_input.output.value,  
4   fn(_, _) { True },  
5 )?
```

 Aiken

The helper applies the custom predicate only to lovelace and still compares the remaining assets exactly. An always-true predicate therefore makes the validator indifferent to lovelace direction without relaxing protocol-parameters NFT or other non-ADA conservation. The continuing output must still be otherwise valid; this change removes only the validator's input-relative lovelace restriction.

Resolution

The input-relative lovelace restriction was removed from protocol-parameter updates. `protocol_params` now preserves non-ADA assets exactly while allowing lovelace reductions under the existing authorization checks.

[I 102] Fixed Protocol Action Set Limits Upgradability

Level	Severity	Status
1	Informational	Resolved

Category: Design Limitation

Affected Files:

- `lib/types.ak`
- `validators/programmable_logic_base.ak`
- `validators/programmable_logic/params.ak`
- `validators/coordination_spend.ak`
- `validators/issuance_mint.ak`

Description

The protocol represents programmable actions as a fixed three-action set across the protocol parameters, PLB redeemer and dispatcher, and coordination update rules. `BaseSpendRedeemer` defines only `SpendViaTransfer`, `SpendViaThirdParty`, and `SpendViaUnfracking`; `ProgrammableLogicGlobalParams` stores one dedicated credential field for each action; `programmable_logic_base` exhaustively maps those three constructors to those three fields; and `coordination_spend` validates the action credentials through three explicit field checks during a configuration update.

An update can therefore replace the validator credential behind an existing action, but it cannot add another protocol action through the parameters. Adding an action requires changing the parameter schema, `BaseSpendRedeemer`, the PLB dispatcher, and the coordination validator. Because `prog_logic_cred` cannot be changed in place, the updated interface would require a new PLB implementation and migration of existing token UTxOs to its address.

Impact

Upgradability is limited to implementations compatible with the existing transfer, third-party, and unfracking action slots. A future protocol-level action must reuse one of those slots or require a new PLB and migration of existing token UTxOs; the three existing actions remain unaffected.

Recommendation

Adopt one of the following architectures:

1. **Generic, data-driven dispatch.** Replace the three individual action-credential fields with a bounded, tagged action list, and replace the fixed `BaseSpendRedeemer` constructors with a generic dispatch redeemer containing `params_idx`, `action_idx`, and `wdr_idx`. During a configuration update, `coordination_spend` should iterate over the list to validate every credential and enforce the list's bounds and identifier rules. On the per-input PLB path, the validator should directly index the selected action using `action_idx` and

compare its credential with the withdrawal at `wdr1_idx`, rather than scanning the action list.

2. **Replaceable `programmable_logic_global` dispatcher.** Reintroduce PLGlobal as the sole protocol delegate required by PLB. Parameterize each PLGlobal deployment with the transfer, third-party, and unfracking script credentials, and replace the three corresponding fields in `ProgrammableLogicGlobalParams` with one mutable `prog_logic_global_cred`. Keep the PLB redeemer independent of action variants. An upgrade can then deploy a new PLGlobal with a different action set or action-redeemer structure and update `prog_logic_global_cred`; existing token UTxOs remain at the same PLB address.

Note: Both options must also consider `issuance_mint`. The issuance policy reads the transfer and third-party credentials from fixed fields in the protocol parameters. It also decodes `TransferRedeemer` and `ThirdPartyRedeemer` to decide whether those validators enforce custody for minted tokens. The upgrade design must account for this dependency.

Resolution

PLB now uses an action-independent redeemer and selects a replaceable `programmable_logic_global` dispatcher through authenticated protocol parameters. Action-specific issuance checks also moved to replaceable `issuance_logic`. Future action changes can therefore update these components without replacing the redesigned PLB or permanent issuance policy.

[I 103] Redundant Fields in Protocol Params

	Level	Severity	Status
	1	Informational	Resolved

Category: Design Redundancy

Affected File: `validators/programmable_logic/params.ak`

Description

In the current protocol parameters, the `ProgrammableLogicGlobalParams` type contains two fields that are redundant and cannot be upgraded: `registry_node_cs` and `prog_logic_cred`. These fields are included in the protocol parameters but do not serve a functional purpose in the upgradability of the protocol.

Impact

Keeping these immutable values in the protocol-parameters datum unnecessarily expands its schema and obscures which parameters can actually be upgraded.

Recommendation

Remove `registry_node_cs` and `prog_logic_cred` from `ProgrammableLogicGlobalParams`. Parameterize the validators that consume these values instead of reading them from the protocol-parameters reference-input datum.

Resolution

The redundant registry and PLB credentials were removed from the protocol-parameters datum. Consumers now receive the required identities as validator parameters; registry spending derives its policy from its own input's payment credential.

[I 104] Redundant nonce parameter in coordination_spend validator

	Level	Severity	Status
	1	Informational	Resolved

Category: Design Simplification

Affected File: `validators/coordination_spend.ak`

Description

The `coordination_spend` validator currently requires a `_nonce` parameter, which is used to generate unique coordination script addresses for each deployment. However, this nonce is not utilized in any validation logic and does not contribute to the upgradability of the protocol.

While the nonce ensures that each deployment has a distinct coordination address, using the nonce to achieve an official coordination address does not provide any additional security or functional benefits.

While the nonce also provides a unique variant to the `protocol_params_mint` validator, the protocol-params policy ID uniqueness can still be achieved by the `utxo_ref` variant that is a parameter to the `protocol_params_mint` validator. The nonce is not necessary for ensuring the uniqueness of the policy ID, as the `utxo_ref` variant can serve this purpose.

So, the coordination address is only a location, and the protocol's real identity is determined by the asset class, which consists of the protocol-params policy ID and the "Protocol-Params" token name. The live UTxO reference changes after every upgrade, while the NFT remains unchanged through value conservation.

It is important to note that using the protocol-params NFT as an authoritative source for the current protocol parameters is critical for ensuring the integrity and authenticity of the protocol's state.

Recommendation

To simplify the validator and reduce unnecessary complexity, it is recommended to remove the `_nonce` parameter from the `coordination_spend` validator. This change will not affect the validation logic or the upgradability of the protocol, as the nonce is not used in any critical operations.

Resolution

The separate coordination validator and its unused nonce were replaced by a combined `protocol_params` minting and spending validator. Its sole `utxo_ref` parameter is used by the one-shot minting check.

[I 105] Static Upgrade-Multisig Configuration

Level	Severity	Status
1	Informational	Resolved

Category: Upgrade Authority / Operational Flexibility

Affected Files:

- `validators/upgrade_multisig.ak`
- `validators/coordination_spend.ak`
- `validators/programmable_logic/params.ak`

Description

The protocol-parameters datum stores `upgrade_cred`, which identifies the authority permitted to update the coordination UTxO. In the intended deployment, it contains the credential of `upgrade_multisig`.

During an upgrade, `coordination_spend` requires a withdrawal from the `upgrade_cred` in the current input datum. `upgrade_multisig` authorizes the transaction when at least `threshold` configured signers appear in `tx.extra_signatories`.

However, `signers` and `threshold` are validator parameters. Changing either value creates a new applied validator instance and credential. Rotating a key, changing membership, or modifying the threshold therefore requires the current multisig to authorize a handover that replaces `upgrade_cred` with the new multisig credential.

The authority is therefore swappable rather than permanently immutable. The current `upgrade_cred` still authorizes the handover, and no coordination-contract or token migration is required. Nevertheless, routine multisig changes require authority reinstantiation rather than a configuration-state update.

Impact

Each signer-set or threshold change requires creating and installing a new multisig instance and completing an authorized `upgrade_cred` handover. This adds operational steps when responding to compromised keys or performing ordinary authority maintenance, increasing handover latency and configuration risk.

Rotation remains possible while the current threshold can still be satisfied. If it can no longer be met, neither the existing design nor an ordinary self-authorized datum update provides recovery without an independent recovery path.

Recommendation

Preferred: Datum-Based Configuration

- Keep a stable `upgrade_multisig` credential in `upgrade_cred`.
- Parameterize it by an authenticated configuration NFT or policy.
- Store the multisig policy in the corresponding inline datum.
- Authorize configuration updates using the old input datum.

- Preserve the configuration NFT, address, and continuing output.
- Enforce valid key hashes, unique signers, bounded configuration size, and `1 <= threshold <= signer_count`.

A simple `{ signers, threshold }` datum is sufficient for flat M-of-N authorization. A native-script-like structure, such as `SundaeSwap-finance/aicone/lib/sundae/multisig.ak`, may be used when nested thresholds, timelocks, or script-based recovery are required.

Alternative: Retain Authority Handover

If retaining the current reference authority:

- Document that signer or threshold changes require a new instance.
- Enforce signer uniqueness and valid threshold bounds.
- Cap the signer count.
- Add deployment and authority-handover validation.
- Consider requiring evidence that the incoming authority is usable.

Recovery

Consider an independent cold, timelocked, or governance recovery branch for cases where the normal multisig threshold can no longer be met.

Resolution

Signer and threshold rules now reside in an authenticated inline configuration datum. Configuration updates must satisfy the authorization rules in the existing datum and preserve the configuration NFT and address. Routine rotation therefore no longer requires replacing `upgrade_cred`.

[I 106] Fixed Registry Lifecycle Limits Node-Schema Upgrades

Level	Severity	Status
 1	Informational	Acknowledged

Category: Design Limitation

Affected Files:

- `validators/registry_spend.ak`
- `validators/registry_mint.ak`
- `validators/issuance_mint.ak`
- `validators/coordination_spend.ak`
- `lib/linked_list.ak`
- `lib/registry_node.ak`

Description

The current protocol upgrade mechanism does not cover the registry lifecycle. `registry_mint` and `registry_spend` are fixed to the current `RegistryNode` structure and its registration and update rules, while `coordination_spend` requires `registry_node_cs` to remain unchanged during protocol-parameter updates.

This becomes a limitation when a protocol upgrade requires new per-policy configuration or different registry rules. For example, adding a protocol action may require each registered policy to specify a script that validates that action. The unfracking action required this type of change: `unfracking_logic_script` was added to `RegistryNode`, together with registration and update checks for the new field.

`registry_mint` decodes the covering node and both resulting nodes as the current `RegistryNode` and applies the current insertion rules. `registry_spend` requires an updated node to remain at the existing registry address and applies `is_field_updated_registry_node`. That helper preserves `key`, `next`, and `minting_logic_script`, and permits changes only to `transfer_logic_script`, `third_party_transfer_logic_script`, `unfracking_logic_script`, and `global_state_cs`. The update must also be authorized by the existing `minting_logic_script`.

Consequently, a protocol upgrade that needs additional per-policy data or different registration or update rules also requires the registry lifecycle to evolve. The current contracts provide no path for replacing that lifecycle or migrating its nodes to a new validator and representation.

Impact

Protocol upgrades that require new registry-backed per-policy data or different registration or update rules cannot be applied through the current upgrade mechanism:

- **Existing registrations:** Existing nodes cannot adopt an incompatible structure or different transition rules through the normal update path.
- **New registrations:** New nodes created under the existing registry remain restricted to the current structure and insertion rules.

- **Permanent consumers:** Existing `issuance_mint` policies decode the complete current `RegistryNode`, so an incompatible representation would also affect issuance for existing tokens.

Upgrades that only replace the values of the four currently mutable fields remain supported. The limitation applies when an upgrade exceeds the current registry interface.

Recommendation

Implement the following design goals and recommendations to address the limitations of the current registry upgrade mechanism:

1. **Registry Upgrade Mechanism:** Extend the protocol upgrade mechanism to cover the full registry lifecycle. This should include support for upgrading the rules for registering new nodes and updating existing nodes.
2. **Registry Update Path:** Extend the current registry update path to allow for node-schema upgrades. This should include an authorized path for migrating existing nodes when a protocol upgrade requires a different datum structure.

Resolution

The client decided to retain the existing registry lifecycle and node schema.

Registry-dependent issuance checks moved out of the permanent policy and into replaceable `issuance_logic` for tokens issued under the redesigned policy.

The unchanged registry can continue registering new tokens and updating existing nodes after a compatible replacement of the upgradeable protocol layer. This requires preserving the existing registry structure and rules, and the permanent issuance interfaces. The issuance template remains usable because the permanent policy references the same protocol-parameters NFT and selects the current issuance logic at runtime.

However, registration and existing-node updates remain tied to the current `RegistryNode` structure and fixed transition rules. No authorized migration to an incompatible node schema or replacement registry lifecycle is implemented; redirecting consumers to another registry does not migrate existing node state.

[I 107] Fixed Issuance Policies Limit Upgrades for Existing Tokens

	Level	Severity	Status
	1	Informational	Resolved

Category: Design Limitation

Affected File: `validators/issuance_mint.ak`

Description

The `issuance_mint` policy currently contains logic that depends on protocol interfaces and rules that may need to change during an upgrade. However, a token's `issuance_mint` policy cannot be replaced without changing that token's policy ID. This creates a limitation for upgrades of existing tokens.

In the `RefInput` path, `issuance_mint` reads the current `transfer_cred` and `third_party_cred` from the protocol parameters. It looks for withdrawal redeemers under those credentials and decodes any matching redeemer as the current `TransferRedeemer` or `ThirdPartyRedeemer` type. A matching `TokenExists { node_idx }` proof or `registry_node_idx` tells `issuance_mint` to trust that validator's custody checks and skip its local `no_escape` check.

If a transaction invokes an upgraded transfer or third-party validator with an incompatible redeemer format, the existing issuance policy cannot decode it, so validation fails before the local custody fallback can run. Even if the redeemer still decodes, if the matching field no longer guarantees that custody was checked, the existing issuance policy may skip `no_escape` on an invalid assumption. Additionally, the policy decodes the current `RegistryNode` type and directly enforces the current custody and PLB output-shape rules. Existing policies cannot adopt incompatible changes to those rules.

Impact

The fixed issuance logic limits the upgrades that existing token policies can support. Incompatible changes to the transfer or third-party redeemers, the `RegistryNode` datum, or issuance-level custody rules can block affected minting and burning transactions or require the protocol to preserve legacy behavior. If a redeemer remains decodable but no longer proves that custody was checked, `issuance_mint` may instead skip `no_escape` without the expected delegated check.

Recommendation

To mitigate the limitations imposed by fixed issuance policies, consider implementing a more flexible approach that allows for seamless upgrades of existing token policies without requiring changes to their policy IDs.

To achieve this, the following design goals and recommendations are proposed:

1. **Design Goal:** Keep only stable, permanent checks in `issuance_mint`. Move every check that may change with the protocol into a separate issuance-logic script that can be replaced through the protocol parameters.
2. **Permanent Issuance Policy:** Parameterize it only with `minting_logic_cred` and `params_policy`. Require the token-specific `minting_logic_cred` withdrawal, authenticate the protocol-parameters NFT, read `issuance_logic_cred` from the authenticated parameters, and require that script's withdraw-zero invocation. Ensure a stable proof that the delegated script validated `own_policy`. Avoid decoding `TransferRedeemer`, `ThirdPartyRedeemer`, `RegistryProof`, or the full `RegistryNode` datum in the permanent policy.
3. **Upgradeable Issuance-Logic Script:** Perform the registry, custody, and PLB output checks that may change between protocol versions. Decode the transfer, third-party, proof, and registry data used by the current protocol version. Update `issuance_logic_cred` together with any protocol change that requires different issuance checks.
4. **Protocol Parameters:** Update the protocol parameters to include the new issuance logic credentials and ensure that the issuance logic script can be upgraded.

By separating the stable checks from the upgradeable logic, existing tokens can maintain their policy IDs while still being able to adapt to changes in the protocol, ensuring a more robust and flexible system.

Resolution

The permanent `issuance_mint` policy now retains issuer authorization, authenticated selection of the live issuance logic, and own-policy coverage. Version-specific registry, custody, and action-redeemer checks moved into replaceable `issuance_logic`, selected through `issuance_logic_cred`. This allows coordinated logic upgrades without changing policy IDs for tokens issued with the redesigned policy; it does not retroactively replace policies instantiated from the pre-redesign code.

End of Report



No Witness Labs — Confidential

Copyright © 2026